Wyższa Szkoła Ekonomii
i Informatyki w Krakowie

Laboratoria, 21 marca 2026r.

Architektura Aplikacji w Pythonie

[Lab 2] Programowanie współbieżne i równoległe w Pythonie

Michał Madejski

Kraków, 2026

Agenda

1. **Wprowadzenie** (definicje, cykl życia wątku, współbieżność a równoległość, asynchroniczność, zestawienie, Operacje I/O-bound, Operacje CPU-bound, GIL, zastosowania)
2. **Threading** (biblioteka, tworzenie wątku, wątki daemon, join, praca z wieloma wątkami, ThreadPoolExecutor, hazard, Lock, Deadlock, Problem producenta i konsumenta, Queue, Semaphore, Timer, Barrier, zadania)
3. **Multiprocessing** (biblioteka, Process, Queue, Pipe, Manager, Pool, zadania)

WSEI

Definicje - przypomnienie

Proces *(process)* jest instancją wykonywanego programu. Proces ma zasoby i wykonuje instrukcje programu.

W Pythonie każdy process jest w rzeczywistości osobną instancją interpretera.

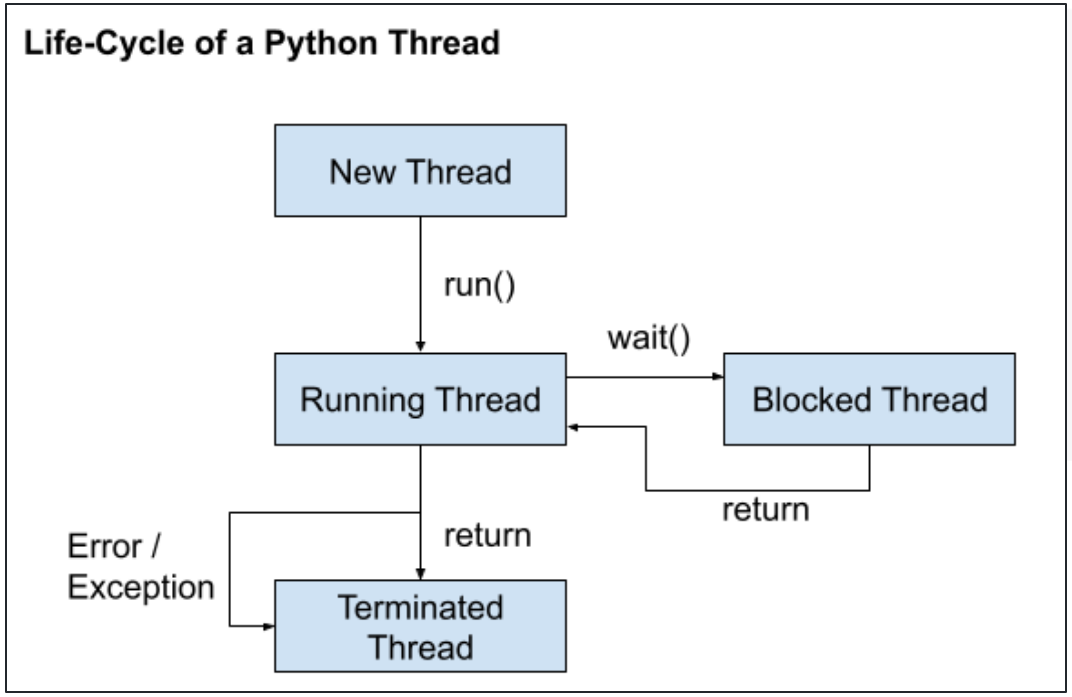
Wątek *(thread)* to osobna jednostka wykonawcza działająca w obrębie tego samego procesu. Oznacza to, że najprostsze programy zawsze mają proces i co najmniej jeden wątek.

Wątki w Pythonie są rzeczywistymi wątkami, a nie symulowanymi wątkami programowymi (jak np. *green threads* w Java)

WSEI

Cykl życia wątku - przypomnienie

Wątek jest uruchomiony gdy interpreter Python zażąda utworzenia nowego wątku natywnego.



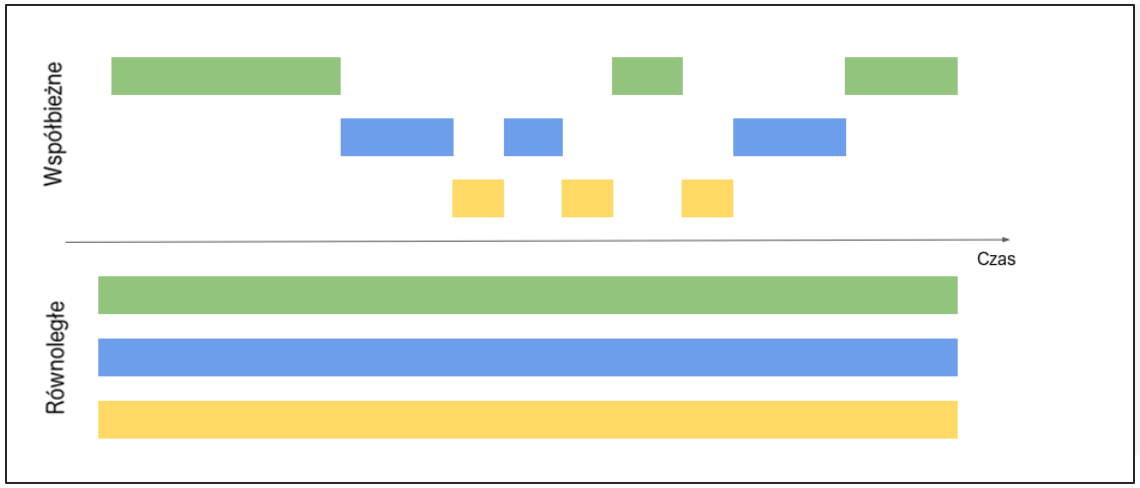
Wątek posiada stany:
- Utworzenia
- Wykonywania
- (opcjonalnie) zablokowania
- Zakończenia

Źródło:
https://superfastpython.com/threading-in-python

WSEI

Współbieżność a równoległość

Równoległość *(multiprocess)* odnosi się do wykonywania wielu zadań w tym samym czasie, zazwyczaj za pomocą wielu procesorów lub rdzeni procesora.



Współbieżność *(concurrency)* odnosi się do techniki organizacji programu, która dopuszcza istnienie wielu wątków. Działają jako przeplecione ze sobą.

W Pythonie, podstawowym modułem do programowania współbieżnego jest *threading*, natomiast do programowania równoległego *multiprocessing*.

Źródło:
https://www.pyjazz.pl/programowanie-wspolbiezne-czesc-1/

WSEI

Asynchroniczność

Programowanie **asynchroniczne** to paradygmat, który umożliwia współbieżne i nieblokujące wykonywanie zadań w aplikacji.

W Pythonie programowanie asynchroniczne można implementować przez moduł *asyncio*.

W *threading* SO może przerwać wątek w dowolnym momencie, aby rozpocząć uruchamianie innego wątku. Nazywa się to **wielozadaniowością z wywłaszczaniem *(preemption).*** Może go przerwać nawet w czasie wykonywania prostych operacji.

Z kolei a*syncio* wykorzystuje **wielozadaniowość kooperacyjną *(cooperative multitasking)***. Zadania muszą ogłaszać, kiedy są gotowe do wyłączenia.

WSEI

Zestawienie

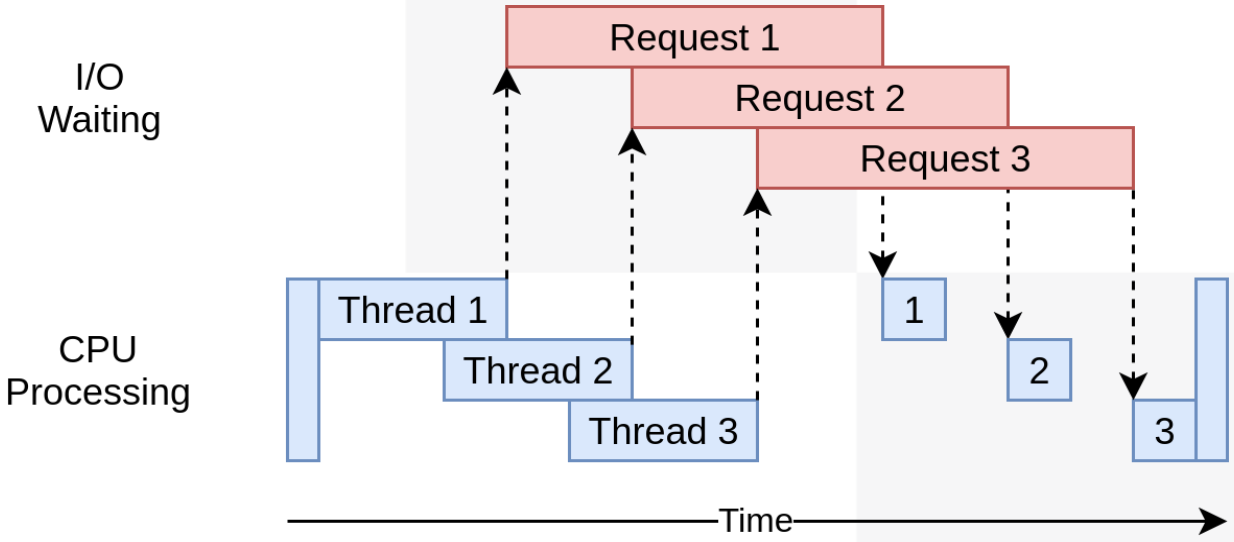
typ	logika	liczba procesorów	biblioteka
Współbieżność (z wywłaszczeniem)	SO decyduje kiedy zmienia wątek.	1	*threading*
asynchroniczność	Wątek ogłasza kiedy jest gotowy do wyłączenia.	1	*asyncio*
Równoległość	Zadania wykonują się w tym samym czasie na różnych procesorach.	wiele	*multiprocessing*

WSEI

Operacje I/O-bound

W **I/O-bound** głównym ograniczeniem wydajnościowym jest **czas oczekiwania na operacje wejścia-wyjścia** (I/O), takie jak czytanie danych z dysku, zapisywanie do pliku, komunikacja sieciowa czy operacje na bazie danych.

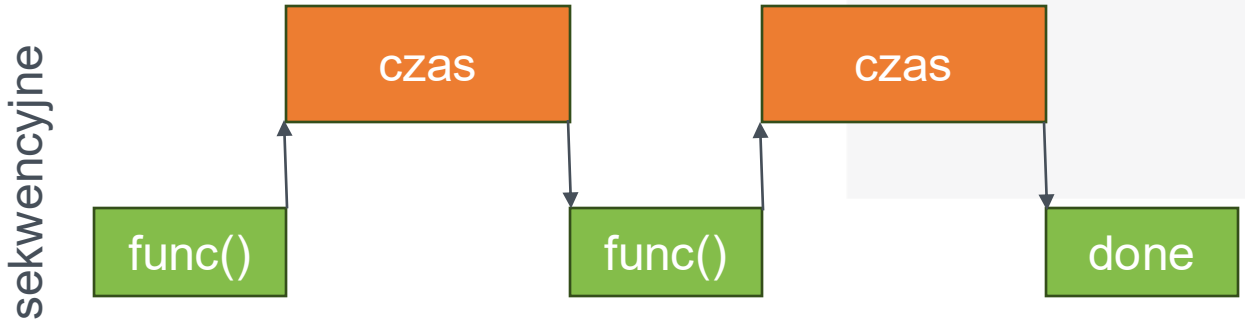
Implementacja współbieżności:

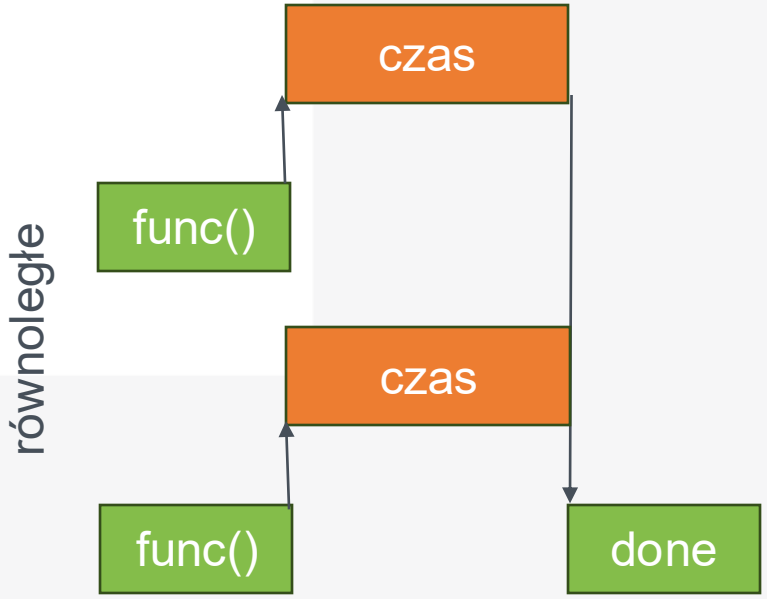


Źródło:
https://realpython.com/python-concurrency/

Operacje CPU-bound

Operacje **CPU-bound** to takie, których głównym ograniczeniem wydajnościowym jest **czas obliczeń**. Wydajność może być poprawiona poprzez zrównoleglenie obliczeń, czyli wykorzystanie wielu rdzeni procesora do równoczesnego wykonywania zadań.





WSEI

Główne zastosowania

Współbieżność i asynchroniczność:

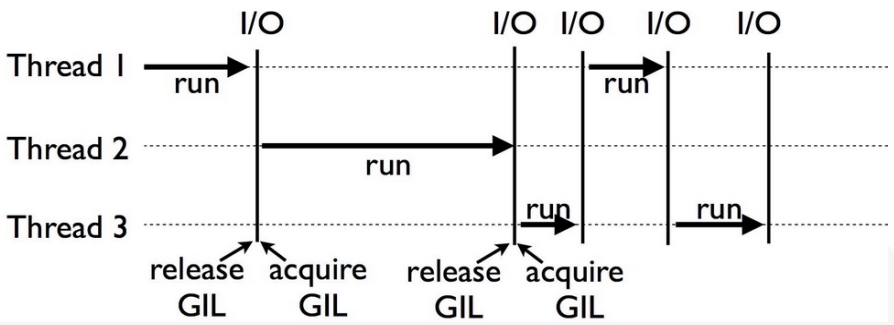
- Serwery sieciowe
- Interaktywne aplikacje
- Integracja z Bazami Danych

Równoległość:

- Operacje na dużych zbiorach danych
- Obliczenia numeryczne

WSEI

GIL



Python **Global Interpreter Lock lub GIL** jest blokadą (lock), która pozwala tylko jednemu wątkowi kontrolować interpreter Pythona, co eliminuje potrzebę synchronizacji dostępu do wrażliwych obszarów pamięci.

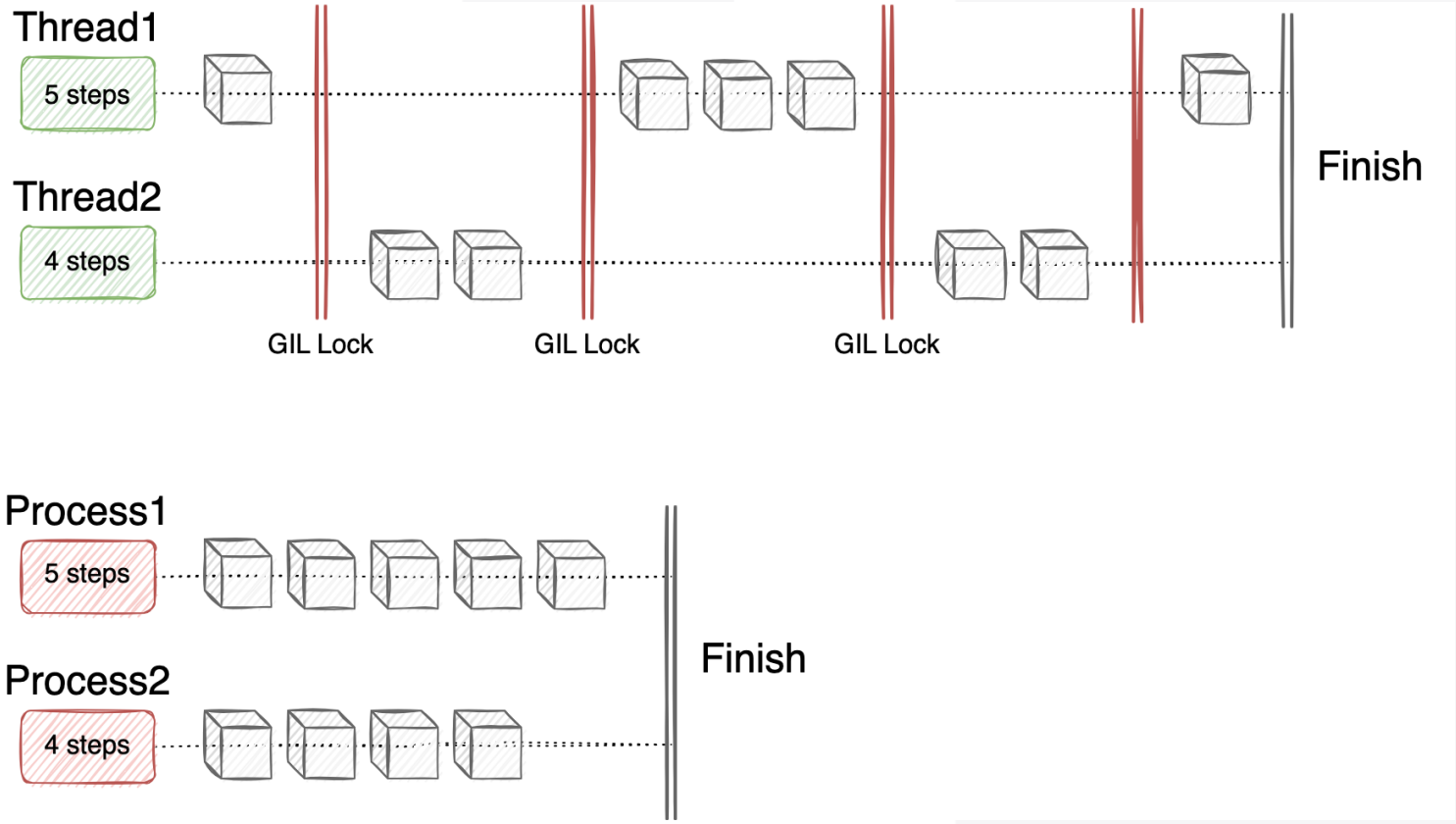
GIL to pojedyncza blokada na samym interpreterze, która wprowadza zasadę, że wykonanie dowolnego kodu Pythona wymaga uzyskania blokady interpretera. Zapobiega to zakleszczeniom.

Można kompensować utratę korzyści z wydajności pojedynczego wątku wynikającą z GIL, dodając inne funkcje zwiększające wydajność, takie jak kompilatory JIT

Więcej informacji:
https://realpython.com/python-gil/

WSEI

GIL


Thread1
5 steps
Thread2
4 steps
GIL Lock
GIL Lock
GIL Lock
Finish
Process1
5 steps
Process2
4 steps
Finish

WSEI

GIL – Python 3.14 (2025-10-07 release)

https://towardsdatascience.com/python-3-14-and-the-end-of-the-gil/

Python 3.14 and the End of the GIL

Exploring the opportunities and challenges of a GIL-free Python

Thomas Reid

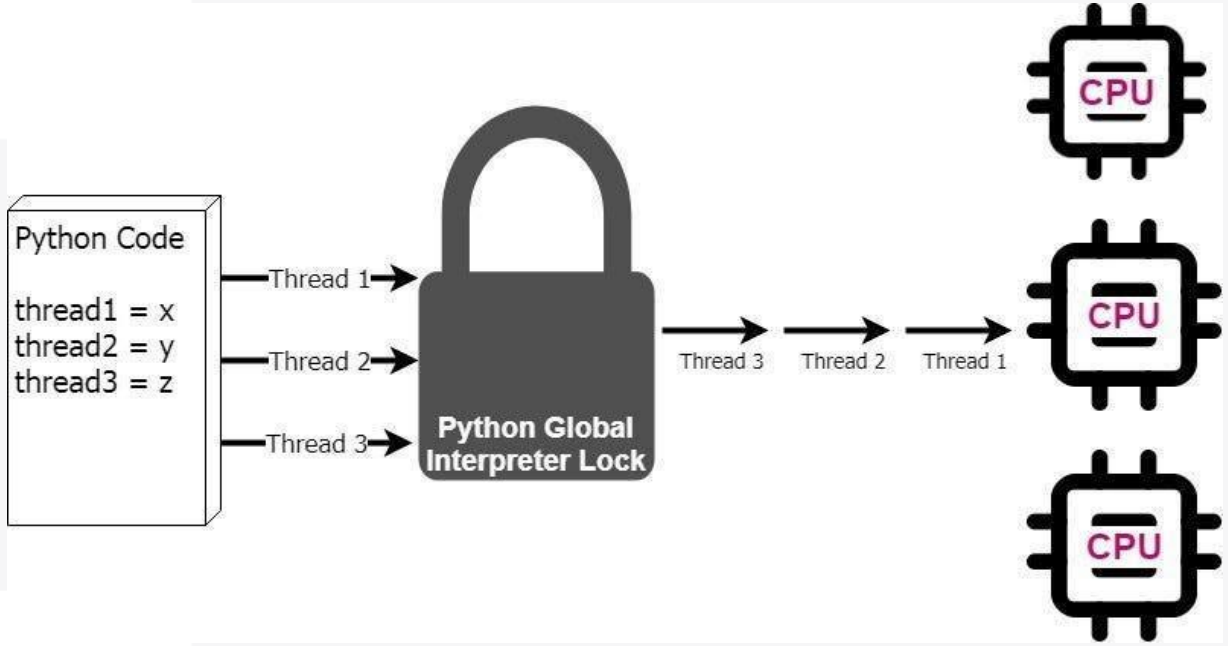
Oct 18, 2025 • 16 min read

WSEI

Faktyczny koniec GIL??

Został zatwierdzony PEP 703 („Making the GIL optional in CPython”), co pozwala na budowanie interpretera bez GIL-a jako **opcję kompilacji**. Oznacza to, że wersja „free-threaded” (bez GIL) staje się wspierana oficjalnie, ale wciąż jako wariant interpretera.

Tryb bez GIL-a jest nie domyślny, ale można teraz użyć interpretera z wyłączonym GIL-em (budowanego z *--disable-gil*).



WSEI

Konsekwencje

- Większe ryzyko race condition
- Większe wymagania bezpieczeństwa wątkowego w standardowych strukturach (fine-grained locks)
- Breakage ekstencji w C / C++ (np. NumPy, Pandas) - Rozszerzenia do tej pory polegały na GIL-u jako mechanizmie synchronizacji. **Free-threaded Python nie będzie działał z rozszerzeniami, które zakładają istnienie GIL-a.**
- Wyższe zużycie pamięci (pojedynczy obiekt ma większy narzut pamięci, wiele struktur ma dodatkowe locki) - ok. +10–20% RAM
- Uruchamianie bez GIL wymaga bardziej złożonych i precyzyjnych mechanizmów blokowania wewnętrznych struktur danych Pythona. Wprowadza to niewielki spadek wydajności dla kodu jednowątkowego, który według szacunków jest o około 5-10% wolniejszy w Pythonie 3.14 w porównaniu ze standardową kompilacją GIL.

WSEI

Konsekwencje

r/Python • 28 dni temu
Active-Fuel-49

GIL free and thread safety

Discussion

For Python 3.14 free GIL version to be usable, shouldn't also Python libraries be re-written to become thread safe? (or the underlying C infrastructure)

98 24 Udostępnij

DivineSentry • 28 dni temu

https://py-free-threading.github.io/tracking/

They do need to be rewritten, though there's an org that has been funding people to add compatibility to the most popular packages (quantsigh) so thankfully, the support is pretty good

88 Odpowiedz Nagródź Udostępnij ...

https://py-free-threading.github.io/tracking/

WSEI

Jak pozbyć się GIL?

- **Zainstaluj wymagania**: Upewnij się, że masz potrzebne narzędzia kompilacyjne (kompilator C, taki jak GCC lub Clang, `make`, itp.).
- **Pobierz źródła**: Pobierz kod źródłowy Pythona 3.14 ze strony oficjalnej lub sklonuj repozytorium CPython z GitHub.
- **Skonfiguruj kompilację**: To najważniejszy krok. W katalogu ze źródłami uruchom skrypt `configure` z flagą `--disable-gil`.

./configure --disable-gil

- **Kompilacja i instalacja**: Uruchom `make` i `make install` tak jak zwykle.

make -j$(nproc)

sudo make install

WSEI

Szybciej bez GIL

The Power of Parallelism: A Code Demonstration

The real magic of free-threaded Python is that it can dramatically speed up CPU-bound multithreaded code **without any changes to the code itself**.

Let's consider a simple, CPU-intensive task:

```
# cpu_bound_task.py
import time
import threading

def heavy_calculation(n):
    """A function that simulates a CPU-bound task."""
    total = 0
    for i in range(n):
        total += i * i

def main():
    num_threads = 4
    calculations_per_thread = 20_000_000
    threads = []

    start_time = time.time()

    for _ in range(num_threads):
        thread = threading.Thread(target=heavy_calculation, args=(calcula
        threads.append(thread)
        thread.start()

    for thread in threads:
        thread.join()

    end_time = time.time()
    print(f"Execution time: {end_time - start_time:.2f} seconds")

if __name__ == "__main__":
    main()
```

Running with the Standard (GIL-enabled) Python:

When you run this script with a standard Python 3.14 interpreter, the GIL ensures only one thread can execute at a time. The threads will run sequentially, not in parallel.

```
$ python3.14 cpu_bound_task.py
Execution time: 5.82 seconds
```

Running with Free-Threaded Python:

Now, run the exact same script with your new free-threaded interpreter.

```
$ python3.14t cpu_bound_task.py
Execution time: 1.51 seconds
```

The results are dramatic. We see a nearly 4x speedup, demonstrating that the threads are now running in true parallel across multiple CPU cores. This is the core benefit of the no-GIL build.

WSEI

Threading

Moduł ***threading*** w Pythonie jest wbudowanym modułem, który umożliwia programowanie w ielowątkowe.

Moduł implementuje funkcjonalności:

- Tworzenia wątków
- Zarządzania wątkami
- Synchronizacji
- Obsługi wyjątków

Dokumentacja:

https://docs.python.org/3/library/threading.html

WSEI

Tworzenie wątku

Moduł ***threading*** umożliwia tworzenie nowych wątków za pomocą klasy ***Thread.***

Należy określić docelową funkcję wykonywaną w wątku ***(target)***. Funkcja wykonywana wątku może przyjmować argumenty, mogą być przekazane w postaci tuple do argumentu „args” konstruktora klasy *Thread* (lub jako słownik do argumentu „kwargs”). Wątek wywołuje się poprzez metodę

```
3   #Uruchamianie wątku
4   import logging
5   import threading
6   import time
7
8   def funkcja_watku(nazwa):
9       logging.info("Watek %s: rozpoczal sie", nazwa)
10      time.sleep(2)
11      logging.info("Watek %s: zakonczyl sie", nazwa)
12
13  if __name__ == "__main__":
14      logging.basicConfig(level=logging.INFO)
15      logging.info("Main: Przed stworzeniem watku")
16
17      x = threading.Thread(target=funkcja_watku, args=('pierwszy',))
18      logging.info("Main: Przed uruchomieniem watku")
19      x.start()
20
21      logging.info("Main: Zakonczenie programu")
```

```
INFO:root:Main: Przed stworzeniem watku
INFO:root:Main: Przed uruchomieniem watku
INFO:root:Watek pierwszy: rozpoczal sie
INFO:root:Main: Zakonczenie programu
INFO:root:Watek pierwszy: zakonczyl sie
```

Tworzenie wątku

Uruchomienie wątku

WSEI

Wątki Daemon

W wyniku na poprzednim slajdzie Wątek zakończył się po zakończeniu programu głównego:

```
INFO:root:Main: Przed stworzeniem watku
INFO:root:Main: Przed uruchomieniem watku
INFO:root:Watek pierwszy: rozpoczal sie
INFO:root:Main: Zakonczenie programu
INFO:root:Watek pierwszy: zakonczyl sie
```

Wątki ***Daemon*** w Pythonie są zamykane automatycznie, gdy główny wątek programu zakończy swoje działanie. Jeśli program uruchamia wątki, które nie są wątkami daemon, to program będzie oczekiwać, aż te wątki zakończą swoje działanie przed zakończeniem pracy programu.

```
x = threading.Thread(target=funkcja_watku, args=('pierwszy',), daemon=True)
logging.info("Main: Przed uruchomieniem watku")
x.start()
```

W informatyce **Demon** definiuje się jako program lub proces, wykonywany wewnątrz środowiska wielozadaniowego systemu operacyjnego, bez konieczności interakcji z użytkownikiem (działający w tle).

WSEI

Join

join(timeout=None) - ta metoda blokuje wątek wywołujący, aż wątek, którego metoda join() została wywołana, zakończy działanie albo do momentu wystąpienia opcjonalnego limitu czasu.

```python
#Uruchamianie wątku
import logging
import threading
import time

def funkcja_watku(nazwa):
    logging.info("Watek %s: rozpoczal sie", nazwa)
    time.sleep(4)
    logging.info("Watek %s: zakonczyl sie", nazwa)

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)
    logging.info("Main: Przed stworzeniem watku")

    x = threading.Thread(target=funkcja_watku, args=('pierwszy',))
    logging.info("Main: Przed uruchomieniem watku")
    x.start()
    x.join()
    logging.info("Main: Zakonczenie programu")
```

join blokuje program główny do zakończenia wątku x

```
INFO:root:Main: Przed stworzeniem watku
INFO:root:Main: Przed uruchomieniem watku
INFO:root:Watek pierwszy: rozpoczal sie
INFO:root:Watek pierwszy: zakonczyl sie
INFO:root:Main: Zakonczenie programu
```

WSEI

Praca z wieloma wątkami

```python
#Praca z wieloma wątkami
import logging
import threading
import time

def funkcja_watku(numer):
    logging.info("Watek %d: rozpoczal sie", numer)
    time.sleep(2)
    logging.info("Watek %d: zakonczyl sie", numer)

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)
    logging.info("Main: Przed stworzeniem watku")

    watki = list()
    for index in range(3):
        logging.info("Main: Utworzenie i start wątku %d.", index)
        x = threading.Thread(target=funkcja_watku, args=(index,))
        watki.append(x)
        x.start()

    for index, watek in enumerate(watki):
        logging.info("Main: Przed join watku %d.", index)
        watek.join()
        logging.info("Main: watek %d zakonczony", index)
```

```
INFO:root:Main: Przed stworzeniem watku
INFO:root:Main: Utworzenie i start wątku 0.
INFO:root:Watek 0: rozpoczal sie
INFO:root:Main: Utworzenie i start wątku 1.
INFO:root:Watek 1: rozpoczal sie
INFO:root:Main: Utworzenie i start wątku 2.
INFO:root:Watek 2: rozpoczal sie
INFO:root:Main: Przed join watku 0.
INFO:root:Watek 0: zakonczyl sie
INFO:root:Main: watek 0 zakonczony
INFO:root:Main: Przed join watku 1.
INFO:root:Watek 1: zakonczyl sie
INFO:root:Watek 2: zakonczyl sie
INFO:root:Main: watek 1 zakonczony
INFO:root:Main: Przed join watku 2.
INFO:root:Main: watek 2 zakonczony
```

Kolejność, w jakiej są uruchamiane wątki, jest określana przez system operacyjny i może być dość trudna do przewidzenia. Może się różnić przy każdym uruchomieniu

WSEI

ThreadPoolExecutor

ThreadPoolExecutor to część modułu concurrent.futures w Pythonie, która zapewnia interfejs do tworzenia i zarządzania **pulą wątków** dla równoległego wykonywania zadań.

Określamy rozmiar puli wątków podczas tworzenia instancji ThreadPoolExecutor.

Możemy przekazać funkcje lub wyrażenia do wykonania do puli wątków za pomocą metody **submit()** lub **map().**

ThreadPoolExecutor automatycznie zarządza cyklem życia wątków w puli, takimi jak startowanie, zatrzymywanie i ponowne uruchamianie

Zaleca się korzystanie z ThreadPoolExecutor z użyciem **menedżera kontekstu (*with*)**, aby nie wywoływać *.join()* dla wątków lub *.shutdown()* w samej puli.

WSEI

ThreadPoolExecutor

```python
#Praca z wieloma wątkami
import logging
import concurrent
import time

def funkcja_watku(numer):
    logging.info("Watek %d: rozpoczal sie", numer)
    time.sleep(2)
    logging.info("Watek %d: zakonczyl sie", numer)

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)

    with concurrent.futures.ThreadPoolExecutor(max_workers=3) as executor:
        executor.map(funkcja_watku, range(3))
```

```
INFO:root:Watek 0: rozpoczal sie
INFO:root:Watek 1: rozpoczal sie
INFO:root:Watek 2: rozpoczal sie
INFO:root:Watek 0: zakonczyl sie
INFO:root:Watek 1: zakonczyl sie
INFO:root:Watek 2: zakonczyl sie
```

Kolejność, w jakiej są wykonywane wątki, nie jest zdefiniowana.

mapa z funkcją i zestawem argumentów

Ilość wątków w puli

WSEI

Hazard (Race Conditions)

Race condition to sytuacja, w której wynik działania programu zależy od kolejności lub czasu wykonania różnych wątków lub procesów. W Pythonie, race conditions mogą wystąpić, gdy kilka wątków próbuje równocześnie modyfikować współdzielone zasoby bez odpowiedniej synchronizacji.

```python
import threading
import time

wspoldzielona_zmienna = 0

def increment():
    global wspoldzielona_zmienna
    temp = wspoldzielona_zmienna
    time.sleep(0.01)
    temp += 1
    wspoldzielona_zmienna = temp


watek1 = threading.Thread(target=increment)
watek2 = threading.Thread(target=increment)

watek1.start()
watek2.start()

watek1.join()
watek2.join()

print("Koncowa wartosc wspoldzielonej zmiennej: ", wspoldzielona_zmienna)
```

Wynik:

```
Koncowa wartosc wspoldzielonej zmiennej:  2
```

lub

```
Koncowa wartosc wspoldzielonej zmiennej:  1
```

WSEI

Lock

Lock jest mechanizmem **synchronizacji**, który pozwala kontrolować dostęp wielu wątków do współdzielonych zasobów. Jest obiektem, który działa jak przepustka do zasobów. Każdy wątek, który chce przepustkę, musi poczekać, aż zostanie zwolniona.

```python
import threading
import time

wspoldzielona_zmienna = 0
lock = threading.Lock()

def increment():
    global wspoldzielona_zmienna
    with lock:
        temp = wspoldzielona_zmienna
        time.sleep(0.01)
        temp += 1
        wspoldzielona_zmienna = temp


watek1 = threading.Thread(target=increment)
watek2 = threading.Thread(target=increment)

watek1.start()
watek2.start()

watek1.join()
watek2.join()

print("Koncowa wartosc wspoldzielonej zmiennej: ", wspoldzielona_zmienna)
```

Metoda ***.acquire()*** na Lock w wątku tworzy "blokadę", a ***.release()*** ją zwalnia.

Lock może działać jako menadżer konteksu, więc można go używać z ***with***.

Wynik:

```
Koncowa wartosc wspoldzielonej zmiennej:  2
```

WSEI

Deadlock

Deadlock (zakleszczenie) to sytuacja, gdy dwa lub więcej wątków są zablokowane wzajemnie i oczekują na siebie nawzajem, aby zwolnić zasoby, które trzymają, przed kontynuacją swojego działania.

```python
import threading

l = threading.Lock()
print("Przed pierwszym zablokowaniem")
l.acquire()
print("Przed drugim zablokowaniem")
l.acquire()
print("Po zablokowanich")
```

```
Przed pierwszym zablokowaniem
Przed drugim zablokowaniem
```

W rezultacie żaden z wątków lub procesów nie może kontynuować pracy, co prowadzi do zatrzymania się całego systemu.

WSEI

Problem producenta i konsumenta

W problemie występują dwa rodzaje wątków: **producent** i **konsument**, którzy dzielą wspólny **bufor**. Zadaniem producenta jest wytworzenie produktu, umieszczenie go w buforze i rozpoczęcie pracy od nowa. W tym samym czasie konsument ma pobrać produkt z bufora. Problemem jest taka synchronizacja procesów, żeby producent nie dodawał nowych jednostek gdy bufor jest pełny, a konsument nie pobierał gdy bufor jest pusty.

W Pythonie problem producenta i konsumenta można rozwiązać przy użyciu różnych mechanizmów synchronizacji, takich jak:

- Lock
- kolejki (Queue)
- zmienne warunkowe,
- semafory (Semaphore)

WSEI

Producent i konsument z użyciem Lock

```python
import threading
import logging
import concurrent.futures

class Pipeline:
    """
    Klasa umożliwiająca jednoelementową kolejkę pomiędzy producentem
    a konsumentem.
    """
    def __init__(self):
        self.wiadomosc = 0
        self.blokada_producenta = threading.Lock()
        self.blokada_konsumenta = threading.Lock()
        self.blokada_konsumenta.acquire()

    def pobierz_wiadomosc(self, nazwa):
        logging.debug("%s: przed pobraniem blokady od konsumenta", nazwa)
        self.blokada_konsumenta.acquire()
        logging.debug("%s: konsument otrzymał blokadę", nazwa)
        wiadomosc = self.wiadomosc
        logging.debug("%s: przed zwolnieniem blokady producenta", nazwa)
        self.blokada_producenta.release()
        logging.debug("%s: blokada producenta została zwolniona", nazwa)
        return wiadomosc

    def ustaw_wiadomosc(self, wiadomosc, nazwa):
        logging.debug("%s: przed pobraniem blokady producenta", nazwa)
        self.blokada_producenta.acquire()
        logging.debug("%s: producent otrzymał blokadę", nazwa)
        self.wiadomosc = wiadomosc
        logging.debug("%s: przed zwolnieniem blokady konsumenta", nazwa)
        self.blokada_konsumenta.release()
        logging.debug("%s: blokada konsumenta została zwolniona", nazwa)
```

Dzięki użyciu klasy Pipeline z mechanizmem **Lock**, zapewnione jest bezpieczne współdzielenie danych między wątkami.

```python
def producent(pipeline):
    for i in range(5):
        logging.info("Producent tworzy wiadomość %s", i)
        pipeline.ustaw_wiadomosc(i, "Producent")

def konsument(pipeline):
    for i in range(5):
        message = pipeline.pobierz_wiadomosc("Konsument")
        logging.info("Konsument otrzymuje wiadomość %s", message)

if __name__ == "__main__":
    logging.basicConfig(level=logging.DEBUG)
    pipeline = Pipeline()

    with concurrent.futures.ThreadPoolExecutor(max_workers=2) as executor:
        executor.submit(producent, pipeline)
        executor.submit(konsument, pipeline)
```

WSEI

Queue

W Pythonie klasa ***Queue*** z modułu ***queue*** umożliwia tworzenie kolejek FIFO.

```python
import queue
import threading
import logging
import time

def producent(q):
    for i in range(5):
        item = f'Wiadomość {i}'
        q.put(item)
        logging.info(f'Dodano do kolejki: {item} \n')
        time.sleep(1)

def konsument(q):
    while True:
        item = q.get()
        logging.info(f'Pobrano z kolejki: {item} \n')
        q.task_done()

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)
    q = queue.Queue()

    threading.Thread(target=producent, args=(q,)).start()
    threading.Thread(target=konsument, args=(q,)).start()
```

Najważniejsze metody w klasie *Queue* to:

- ***put(item)***: Dodaje element do kolejki.
- ***get():*** Pobiera element z kolejki, usuwając go z kolejki.
- ***empty()***: Zwraca *True*, jeśli kolejka jest pusta.
- ***full()***: Zwraca *True*, jeśli kolejka jest pełna.
- ***qsize()***: Zwraca liczbę elementów w kolejce.

WSEI

Semaphore

Semafor (Semaphore) pozwala na kontrolowanie ilości dostępnych zasobów i zapewnia, że tylko określona liczba wątków lub procesów może uzyskać dostęp do zasobów jednocześnie. Semafor zapewnia liczbę dostępnych "przepustek", które określają, ile wątków może jednocześnie uzyskać dostęp do chronionego zasobu.

```python
import threading
import time
import logging

semaphore = threading.Semaphore(3)

def watek(numer):
    logging.warning(f'Wątek {numer} próbuje uzyskać dostęp')
    with semaphore:
        logging.warning(f'Wątek {numer} uzyskał dostęp')
        time.sleep(2)
        logging.warning(f'Wątek {numer} zwalnia dostęp')

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)

    for i in range(6):
        threading.Thread(target=watek, args=(i,)).start()
```

```
WARNING:root:Wątek 0 próbuje uzyskać dostęp
WARNING:root:Wątek 1 próbuje uzyskać dostęp
WARNING:root:Wątek 0 uzyskał dostęp
WARNING:root:Wątek 2 próbuje uzyskać dostęp
WARNING:root:Wątek 3 próbuje uzyskać dostęp
WARNING:root:Wątek 1 uzyskał dostęp
WARNING:root:Wątek 4 próbuje uzyskać dostęp
WARNING:root:Wątek 2 uzyskał dostęp
WARNING:root:Wątek 5 próbuje uzyskać dostęp
WARNING:root:Wątek 2 zwalnia dostęp
WARNING:root:Wątek 1 zwalnia dostęp
WARNING:root:Wątek 0 zwalnia dostęp
WARNING:root:Wątek 3 uzyskał dostęp
WARNING:root:Wątek 4 uzyskał dostęp
WARNING:root:Wątek 5 uzyskał dostęp
WARNING:root:Wątek 5 zwalnia dostęp
WARNING:root:Wątek 3 zwalnia dostęp
WARNING:root:Wątek 4 zwalnia dostęp
```

WSEI

Timer

threading.Timer w Pythonie to klasa, która umożliwia wykonywanie określonych funkcji lub kodu w określonym interwale czasowym. Pozwala to na wykonywanie **operacji cyklicznych** lub **opóźnionych** bez blokowania głównego wątku.

```
timer = threading.Timer(5, funkcja)
timer.start()
```

funkcja zostanie uruchomiona 5s po wykonaniu *timer.start()*

WSEI

Barrier

threading.Barrier w Pythonie to klasa synchronizująca, która umożliwia grupie wątków synchroniczne oczekiwanie na siebie nawzajem przed kontynuacją dalszego wykonywania kodu.

```python
import threading
import logging

def zadanie(barrier):
    logging.info("Wątek przed punktem synchronizacji")
    barrier.wait()
    logging.info("Wątek po punkcie synchronizacji")

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)
    barrier = threading.Barrier(3)
    for _ in range(3):
        threading.Thread(target=zadanie, args=(barrier,)).start()
```

```
INFO:root:Wątek przed punktem synchronizacji
INFO:root:Wątek przed punktem synchronizacji
INFO:root:Wątek przed punktem synchronizacji
INFO:root:Wątek po punkcie synchronizacji
INFO:root:Wątek po punkcie synchronizacji
INFO:root:Wątek po punkcie synchronizacji
```

WSEI

DEMO

Kod zawarty w pliku lab2_demo_raw.py umożliwia pobieranie tytułów wydarzeń z kalendarium miasta Krakowa dla wskazanego dnia z 5 kolejnych stron serwisu: https://www.krakow.pl/kalendarium/...

Zmodyfikuj kod z użyciem modułu threading, aby przyspieszyć jego działanie.
W rozwiązaniu wykorzystaj ThreadPoolExecutor do równoległego pobierania danych z wielu stron.

```python
def download_site(url):
    """
    Pobiera treść strony i wyciąga tytuły wydarzeń.
    """
    titles_from_site: list[str] = []
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()

        soup = BeautifulSoup(response.content, 'html.parser')
        # Znajdujemy wszystkie linki do wydarzeń, które zawierają nagłówki h3 z tytułami
        event_links = soup.select('.item__link h3')

        for item in event_links:
            title = item.text.strip()
            logging.info(f"  [OK] Pobrano: {title[:50]}...")
            titles_from_site.append(title)

    except Exception as e:
        logging.error(f"  [BŁĄD] Nie udało się pobrać {url}: {e}")

    return titles_from_site

def download_all_sites(sites):
    """
    Pobiera dane ze wszystkich stron po kolei (sekwencyjnie).
    """
    all_titles = []
    for url in sites:
        logging.info(f"Pobieranie strony: {url}")
        titles = download_site(url)
        all_titles.extend(titles)
    return all_titles

if __name__ == "__main__":
    # Dzisiejsza data dla kalendarium
    date_str = "2026-03-20"
    # Przygotowujemy listę 5 stron kalendarza (indeksy 0-4)
    base_url = "https://www.krakow.pl/kalendarium/1919,shw"
    sites = [f"{base_url},{date_str},{i},day.html" for i in range(5)]

    logging.info(f"=== ROZPOCZĘCIE POBIERANIA SEKWENCYJNEGO (5 stron) ===")
    start_time = time.time()

    all_titles = download_all_sites(sites)

    end_time = time.time()
    duration = end_time - start_time

    logging.info(f"=== ZAKOŃCZONO ===")   Explain and Fix Problem (⇧⌘.)
    logging.info(f"Łącznie pobrano tytułów: {len(all_titles)}")
    print("\nPierwsze 10 tytułów:")
    for i, title in enumerate(all_titles[:10], 1):
        print(f"{i}. {title}")

    logging.info(f"\nCzas wykonania sekwencyjnego: {duration:.2f} sekund")
```

WSEI

Zadania

1. Przy użyciu publicznego API Cat Facts (https://catfact.ninja/fact), które zwraca przy każdym wywołaniu losowy fakt na temat kotów, pobierz sekwencyjnie 20 faktów i zmierz czas całkowitego działania programu. Następnie, zmodyfikuj kod, aby przyśpieszyć działanie kodu. Wykorzystaj ThreadPoolExecutor.

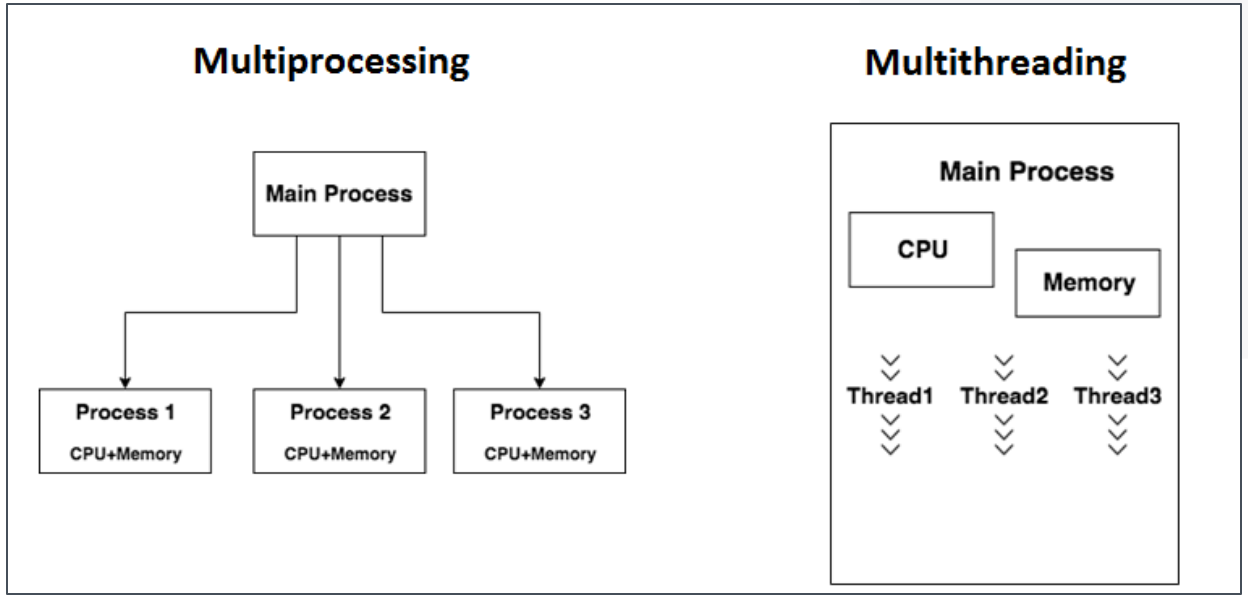
   Protip: Użyj *requests.get(CAT_API_URL).json().get('fact')* aby pobrać fakt

2. Napisz progam o strukturze *producent-consumers*, gdzie producent generuje kolejne liczby naturalne i dodaje je do kolejki. Dwóch konsumentów, jeden pobiera tylko parzyste, drugi tylko nieparzyste liczby z tej kolejki. Użyj wątków do realizacji producenta i obu konsumentów, oraz klasy *queue.Queue* do zarządzania kolejką. Program powinien zakończyć działanie po wygenerowaniu i przetworzeniu określonej liczby liczb naturalnych.

WSEI

Multiprocessing

Multiprocessing odnosi się do zdolności systemu do obsługi więcej niż jednego procesora jednocześnie. Aplikacje w systemie wieloprocesorowym są "podzielone" na mniejsze, które działają niezależnie. System operacyjny przydziela te wątki do procesorów, poprawiając wydajność systemu.



Wiele procesów może być uruchomionych równolegle, ponieważ każdy proces ma własny interpreter, który wykonuje mu przydzielone instrukcje (GIL).

W Pythonie używamy modułu ***multiprocessing***

Źródło:

https://hiteshmishra708.medium.com/multiprocessing-in-python-c6735fa70f3f

Process

Obiekty procesu reprezentują aktywność, która jest uruchamiana w osobnym procesie. Klasa ***Process*** posiada odpowiedniki wszystkich metod klasy ***threading.Thread.***

```python
from multiprocessing import Process
import os

def info(title):
    print(title)
    print('Nazwa modulu:', __name__)
    print('parent process:', os.getppid())
    print('process id:', os.getpid())

def f(name):
    info('function f')
    print('hello', name)

if __name__ == '__main__':
    info('main line')
    p = Process(target=f, args=('wsei',))
    p.start()
    p.join()
```

```
main line
module name: __main__
parent process: 7396
process id: 17456
```

WSEI

Queue

Kolejki (**Queue**) w module **multiprocessing** są używane do bezpiecznego komunikowania się między procesami. Pozwalają one na przekazywanie danych między procesami w sposób, który jest odporny. Działają analogicznie do kolejek ***Queue*** z modułu ***queue.***

```python
from multiprocessing import Process, Queue
import time
import logging

logging.basicConfig(level = logging.INFO)

def producer(queue):
    for i in range(5):
        time.sleep(1)
        item = f'Element {i}'
        queue.put(item)
        logging.warning(f'Produced: {item}')

def consumer(queue):
    while True:
        item = queue.get()
        if item is None:
            break
        logging.warning(f'Consumed: {item}')

if __name__ == '__main__':
    queue = Queue()

    producer_process = Process(target=producer, args=(queue,))
    consumer_process = Process(target=consumer, args=(queue,))
    producer_process.start()
    consumer_process.start()
    producer_process.join()
    queue.put(None)
    consumer_process.join()
```

```
WARNING:root:Produced: Element 0
WARNING:root:Consumed: Element 0
WARNING:root:Produced: Element 1
WARNING:root:Consumed: Element 1
WARNING:root:Produced: Element 2
WARNING:root:Consumed: Element 2
WARNING:root:Produced: Element 3
WARNING:root:Consumed: Element 3
WARNING:root:Produced: Element 4
WARNING:root:Consumed: Element 4
```

WSEI

Pipe

Potoki (***Pipes***) w module ***multiprocessing*** są używane do komunikacji między procesami poprzez przesyłanie strumieni danych. Każdy potok zapewnia dwukierunkową komunikację między dwoma procesami, gdzie jeden proces może pisać do potoku, a drugi proces może czytać z niego.

```
66  from multiprocessing import Process, Pipe
67
68  def f(conn):
69      conn.send("Hello")
70      conn.close()
71
72  if __name__ == '__main__':
73      parent_conn, child_conn = Pipe()
74      p = Process(target=f, args=(child_conn,))
75      p.start()
76      print(parent_conn.recv())
77      p.join()
```

WSEI

Manager

Manager w module *multiprocessing* jest używany do współdzielenia zasobów między procesami. Umożliwia tworzenie zarządców, które są obiektami serwerowymi, zapewniającymi dostęp do współdzielonych obiektów. Dzięki temu obiekty te mogą być używane przez różne procesy, a zmiany dokonane przez jeden proces są widoczne dla pozostałych.

```python
from multiprocessing import Process, Manager

def modify_list(shared_list, index, value):
    shared_list[index] = value

if __name__ == '__main__':
    with Manager() as manager:
        shared_list = manager.list(range(5))

        processes = []
        for i in range(5):
            p = Process(target=modify_list, args=(shared_list, i, i*10))
            processes.append(p)
            p.start()

        for p in processes:
            p.join()

        print("Modified list:", shared_list)
```

WSEI

Pool

Pool w module *multiprocessing* umożliwia równoczesne wykonywanie wielu zadań w puli procesów. Ma identyczne działanie jak ***ThreadPoolExecutor*** z modułu ***concurrent.future.***

```python
from multiprocessing import Pool

def square(x):
    return x * x

if __name__ == '__main__':
    with Pool(processes=4) as pool:
        results = pool.map(square, range(10))
```

WSEI

DEMO

Napisz program, który zrównolegla wyszukiwanie liczb pierwszych w dużym zakresie.

Wykorzystaj moduł multiprocessing, aby utworzyć kilka procesów, z których każdy będzie przetwarzał inne fragmenty zakresu liczb.

```python
import multiprocessing

def is_prime(n):
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    i = 3
    while i * i <= n:
        if n % i == 0:
            return False
        i += 2
    return True

def find_primes(start, end):
    primes = []
    for num in range(start, end + 1):
        if is_prime(num):
            primes.append(num)
    return primes

def main():
    start_range = 1
    end_range = 1000000
    num_processes = 4

    pool = multiprocessing.Pool(num_processes)
    chunk_size = (end_range - start_range) // num_processes
    ranges = [(i, min(i + chunk_size, end_range)) for i in range(start_range, end_range, chunk_size)]

    results = pool.starmap(find_primes, ranges)
    pool.close()
    pool.join()
```

WSEI

Zadanie

Napisz program, który zrównolegli obliczanie sumy kolejnych stu potęg dla każdej liczby z ciągu liczb naturalnych w dużym zakresie.

Wykorzystaj moduł multiprocessing, aby utworzyć kilka procesów, z których każdy będzie przetwarzał inne fragmenty zakresu liczb.

WSEI

Wyższa Szkoła Ekonomii
i Informatyki w Krakowie